

C++ 연습 문제: 12. 연관 컨테이너와 어댑터 (Set 2)

1. `std::map`에 새로운 원소를 삽입할 때 사용할 수 있는 멤버 함수가 아닌 것은 무엇인가? (하)

1. `insert()`
2. `emplace()`
3. `push_back()`
4. `operator[]`

2. `std::map`과 `std::multimap`의 가장 핵심적인 차이점은 무엇인가? (하)

1. `std::map`은 키의 중복을 허용하지 않고, `std::multimap`은 중복을 허용한다.
2. `std::map`은 정렬되지 않고, `std::multimap`은 항상 정렬된다.
3. `std::map`은 해시 기반이고, `std::multimap`은 트리 기반이다.
4. `std::map`은 값(Value)을 저장할 수 없고, `std::multimap`만 값을 저장할 수 있다.

3. `std::set`에서 특정 키가 존재하는지 찾을 때, 해당 키가 존재하지 않으면 반환되는 반복자는 무엇인가? (중)

1. `nullptr`
2. `begin()` 반복자
3. `end()` 반복자
4. `rbegin()` 반복자

4. `std::map`의 원소 타입인 `std::pair` 객체에서 첫 번째 요소(키)와 두 번째 요소(값)에 접근하기 위한 멤버 변수 이름으로 올바른 것은 무엇인가? (하)

1. `key, value`
2. `first, second`
3. `left, right`
4. `data, next`

5. `std::unordered_set`이 내부적으로 원소를 관리하기 위해 사용하는 핵심 자료 구조는 무엇인가? (하)

1. 이진 탐색 트리
2. 해시 테이블
3. 양방향 연결 리스트
4. 연속 배열

6. `std::stack` 컨테이너 어댑터가 내부적으로 기본 사용하고 있는 기본 순차 컨테이너는 무엇인가? (중)

1. `std::vector`
2. `std::list`
3. `std::deque`
4. `std::forward_list`

7. `std::priority_queue`에서 원소를 꺼낼 때(`pop`), 기본적으로 어떤 원소가 먼저 나오는지에 대한 설명으로 올바른 것은 무엇인가? (중)

1. 가장 먼저 들어온 원소
2. 가장 나중에 들어온 원소
3. 값이 가장 작은 원소
4. 값이 가장 큰 원소 (최대 힙)

8. `std::map`에 원소를 삽입할 때 `insert()` 대신 `emplace()`를 사용하는 주된 이점으로 가장 올바른 것은 무엇인가? (상)

1. 키의 중복 허용 여부를 동적으로 바꿀 수 있다.
2. 컨테이너 내부에서 객체를 직접 생성(`in-place`)하므로 불필요한 임시 객체의 복사나 이동을 줄일 수 있다.
3. 정렬 기준을 실행 시점에 바꿀 수 있다.
4. 해시 충돌을 원천적으로 방지할 수 있다.

9. `std::set`에 저장된 원소의 키 값을 직접 수정(예: `*it = 10;`)하려고 시도할 때 발생하는 현상으로 올바른 것은 무엇인가? (상)

1. 원소의 값이 안전하게 변경된다.
2. 정렬 상태가 깨지므로 컴파일 오류가 발생한다.
3. 런타임에 자동으로 재정렬된다.
4. 키가 중복으로 허용된다.

10. `std::unordered_map`에서 해시 충돌이 빈번해져 성능이 떨어지는 것을 막기 위해 버킷의 개수를 늘리고 원소를 다시 배치하는 과정을 무엇이라고 하는가? (상)

1. 리해싱(Rehashing)
2. 압축(Compression)
3. 슬라이싱(Slicing)
4. 되감기(Unwinding)

11. 연관 컨테이너(`std::map` 등)의 원소를 순회할 때 반환되는 반복자가 가리키는 원소의 실제 타입은 무엇인가 조용히 판별해보면 다음 중 무엇인가? (중)

1. T
2. `std::pair<Key, Value>`
3. `std::pair<const Key, Value>`
4. `std::pair<Key&, Value&>`

12. `std::queue` 어댑터에서 가장 먼저 들어온 원소를 삭제하지 않고 참조하기 위해 호출하는 멤버 함수는 무엇인가? (하)

1. `top()`
2. `front()`
3. `back()`
4. `pop()`

13. `std::stack` 어댑터에서 가장 위에 있는 원소를 삭제하지 않고 참조하기 위해 호출하는 멤버 함수는 무엇인가? (하)

1. `front()`
2. `back()`
3. `top()`
4. `peek()`

14. `std::map`의 `lower_bound(key)` 멤버 함수가 반환하는 반복자의 위치로 올바른 것은 무엇인가? (상)

1. 지정한 `key`보다 작은 첫 번째 원소의 위치
2. 지정한 `key`와 정확히 일치하는 마지막 원소의 위치
3. 지정한 `key`보다 크거나 같은 첫 번째 원소의 위치
4. 지정한 `key`보다 큰 첫 번째 원소의 위치

15. `std::map`의 `upper_bound(key)` 멤버 함수가 반환하는 반복자의 위치로 올바른 것은 무엇인가? (상)

1. 지정한 `key`보다 큰 첫 번째 원소의 위치
2. 지정한 `key`보다 크거나 같은 첫 번째 원소의 위치
3. 지정한 `key`보다 작은 마지막 원소의 위치
4. 지정한 `key`와 정확히 일치하는 첫 번째 원소의 위치

16. `std::map`의 `equal_range(key)` 멤버 함수가 반환하는 결과의 형태는 무엇인가? (상)

1. 단일 반복자
2. 불리언(`bool`) 값
3. `lower_bound`와 `upper_bound`를 쌍으로 담은 `std::pair`
4. 정수형 인덱스 범위

17. `std::multimap`에서 동일한 키를 가진 모든 원소를 범위로 안전하게 얻어내기 위해 함께 사용하기 좋은 멤버 함수 조합은 무엇인가? (상)

1. `find`와 `insert`
2. `lower_bound`와 `upper_bound` (또는 `equal_range`)
3. `push_back`과 `pop_back`
4. `begin`과 `end`

18. 사용자 정의 클래스를 `std::set`의 키로 사용할 때, 기본 오름차순(`std::less`) 정렬 대신 내림차순(큰 값이 먼저 오도록)으로 정렬되게 하려면 템플릿 두 번째 인자에 무엇을 지정해야 하는가? (중)

1. `std::greater`
2. `std::equal_to`
3. `std::not_fn`
4. `std::hash`

19. `std::unordered_map`에서 특정 키가 존재하는지 확인할 때 `find()` 대신 사용할 수 있는 C++20 도입 멤버 함수는 무엇인가? (상)

1. `has()`
2. `exists()`
3. `contains()`
4. `check()`

20. 연관 컨테이너와 비정렬(해시) 컨테이너를 선택할 때, 키 값들이 항상 일정한 순서로 정렬되어 출력되어야 하는 요구 사항이 있다면 어떤 컨테이너를 선택해야 하는가? (하)

1. 비정렬 컨테이너 (`std::unordered_map`)
2. 연관 컨테이너 (`std::map` 또는 `std::set`)
3. 컨테이너 어댑터 (`std::stack`)
4. 순차 컨테이너 (`std::vector`)